Revisión crítica: especificación frente al código

Sprints 4 a 6 · 28 de octubre de 2026 · `main` en `2ab0bd6` (CS-09, CS-11, CS-12, CS-13 y CS-26) · Regla 5 de CLAUDE.md

Resumen

Tres revisiones independientes leyeron el código frente a SPEC.md, las reglas AR, los ADR y la documentación: base de datos y seguridad; dominio, dinero y arquitectura del backend; y frontend, infraestructura y documentación. Cada hallazgo se comprobó en el código; los importantes se verificaron otra vez antes de escribir este informe, con pruebas mínimas cuando hacía falta.

0 bloqueantes, 9 importantes y 19 menores. Nada impide seguir con el Sprint 7 (CS-31 y CS-18), pero antes conviene cerrar tres cosas que hoy dan una falsa seguridad: una prueba de arquitectura que no revisa ninguna ruta (I12), un commit que llega después de la respuesta (I13) y un respaldo que avisa "listo" aunque falle (I14).

Seis decisiones son tuyas (regla 2): la fecha de cobro antes del inicio de un contrato futuro (I16), la renovación de contratos (I17), el aviso si el worker se cae si costara algo (I15), las escrituras en el contexto del inquilino, los permisos que sobran a los roles y el hash de IP de las aceptaciones legales. Siguen abiertas las de la revisión anterior: I5, los marcadores legales de I9, las dos políticas de privacidad y la gracia de la prueba gratis.

Lo demás lo resuelvo yo (regla 1) en PR pequeños, en el orden de "Plan propuesto". Ninguno de los hallazgos es explotable hoy: no hay nada desplegado ni datos reales.

Hallazgos importantes

Id	Hallazgo	Corrección	Cuándo
I12	**Deuda.** La prueba de AR-20 no revisa ninguna ruta. Con FastAPI 0.162, `app.routes` guarda un envoltorio por cada router incluido y la prueba solo mira las entradas `APIRoute`: ve `/api/health` y nada de `/api/app`. Con una lista de permitidas vacía devuelve `[]`, aunque 4 rutas no tienen `current_account`. Hoy todas cumplen (revisado a mano), pero la regla no protege nada.	Recorrer los routers incluidos y comparar con las rutas de `app.openapi()`; meta-test con `include_router`; exigir `require_csrf` en los métodos que escriben (cierra la parte de AR-20 de I8).	Ahora · regla 1
I13	**Hueco.** El commit ocurre después de enviar la respuesta. La dependencia de transacción tiene el alcance de la petición, que FastAPI cierra después de `await response()`. El 201 de un pago sale antes de que el pago sea durable: si el commit falla, el cliente ya descartó su id. La pantalla relee el estado de cuenta justo después y puede no ver el pago, y el bloqueo de la fila de la cuenta dura más que la petición.	`Depends(anonymous_transaction, scope="function")` con un solo alias compartido (mezclar alcances abriría dos transacciones, la segunda sin `app.account_id`) y un test de que el commit precede a la respuesta.	Ahora · regla 1
I14	**Hueco.** El respaldo cada 6 horas avisa a Better Stack aunque falle. En el bucle, `back_up || echo` desactiva `set -e` dentro de la función: si `pg_dump`, `age` o `rclone` fallan, igual sube un archivo vacío o truncado y envía el heartbeat. CI solo prueba `--once`, donde `set -e` sí funciona.	Enviar el heartbeat solo tras una subida confirmada (ejecutar `--once` como proceso hijo), validar las variables obligatorias al arrancar y un caso de CI con destino roto que no debe avisar.	Ahora, antes de activar los respaldos · regla 1

I15	**Hueco.** Si el worker o la base se caen, nadie avisa. `/api/health` responde "ok" sin tocar PostgreSQL. Las alertas las envía el propio worker, así que uno caído o colgado no avisa. Un intento matado a la mitad queda en `running` y, tras 3, la cuenta se salta ese día sin fila `failed`. Se detendrían los cargos, las copias de los reclamos (con plazo legal) y, desde CS-34, los recordatorios de impuesto.	`/api/health` con `SELECT 1`; heartbeat del worker en cada vuelta; alerta por intentos en `running` de más de N minutos; reglas de alerta de Sentry documentadas en `server.md`.	Antes de la beta · regla 1 si el plan gratis de Better Stack admite otro heartbeat (por confirmar); si no, regla 2
I16	**Inconsistencia en SPEC.** No se puede registrar un cobro anterior al inicio de un contrato futuro. Si el contrato empieza el 1 de noviembre y el inquilino paga noviembre al firmar, el 28 de octubre, la API rechaza la fecha real porque es anterior al mes de control. El arrendador tendría que fecharlo en noviembre: el tributo pasa a la declaración de noviembre y la de octubre se omite. Contradice "pagos adelantados: cuentan para el impuesto del mes en que se cobraron".	(a) Aceptar cobros desde el 1 de enero de 2026 en cualquier contrato, con un aviso si la fecha es anterior al mes de control (recomendada). (b) Aceptarlos desde el mes anterior al inicio. (c) Mantener la regla.	Antes de la beta · regla 2 (impuesto)
I17	**Hueco.** Un contrato no se puede renovar, pero el worker lo cierra solo en su fecha de fin. SPEC dice "si pasa sin renovación, el worker lo cierra", y ninguna historia permite renovar ni mover la fecha de fin: CS-31 solo termina. La mayoría de contratos vence al año, así que algunos cerrarán durante la beta, dejarán de generar cargos y saldrán del panel. Volver a crearlos parte el historial, cambia el enlace del inquilino y deja el saldo a favor en el contrato viejo.	Recomendado: agregar a CS-31 "extender o editar el fin previsto" (+1 punto; el Sprint 7 pasa de 6 a 7). Preguntar al abogado si, cuando el inquilino sigue después del fin, conviene no cerrar el contrato solo.	Antes de la beta · regla 2 (alcance y puntos)
I18	**Hueco.** El servidor acepta saldos anteriores imposibles, y la base del impuesto depende de que el formulario web los esconda. Por la API, un contrato que empieza en marzo de 2026 puede registrar "deuda de 2025 o antes": un pago de S/ 1,200 en octubre se asigna entero a esa deuda y la base de octubre queda en S/ 0 en vez de S/ 1,200. También pasan una deuda de 2026 sin meses previos y un fin anterior al mes de control.	Validar en el dominio: deuda de 2025 solo si el inicio es anterior a 2026; deuda de 2026 solo si el inicio es anterior al mes de control; fin no anterior al mes de control. Un test por caso. Aplica la definición que SPEC ya tiene.	Ahora, antes de CS-10 · regla 1 (te lo menciono porque toca la base)
I19	**Hueco.** Una respuesta perdida al guardar un pago puede duplicarlo. Si el servidor guarda el pago pero el celular no recibe la respuesta, la pantalla dice "Algo salió mal" sin recargar la lista. Al cancelar y volver a abrir el formulario se genera un id nuevo, y el mismo pago queda dos veces: infla el saldo a favor y la base del 1683. Un 409 (reintento con datos cambiados) y un 401 también se muestran como error genérico.	Ante cualquier error, recargar el estado de cuenta; conservar el id hasta confirmar el guardado; mensajes propios para 409 y 401; test de "falla de red y reintento con el mismo id". Lo mismo para las anulaciones.	Ahora, antes de que CS-31, CS-15 y CS-34 copien el patrón · regla 1

I20	**Hueco.** `CASERO_PRIVACY_INBOX` está decidido y en SPEC, pero no está construido. Caddy no lo sirve en `/api/public/business` y la web no lo reemplaza, así que la política muestra "[correo de privacidad]" en el único canal para ejercer los derechos. Ningún test impide publicar un marcador `[…]` o el aviso de borrador.	Cablear la variable de punta a punta (Caddy, web, compose, `server.md` y test del borde) y un test que falle con marcadores o "Borrador", activable en la puerta de publicación. Los demás marcadores (fecha, expediente, cláusulas) son tuyos.	Antes de publicar la landing · regla 1 el cableado; regla 2 los textos

Hallazgos menores

Datos y seguridad

- Los registros de librerías (logging estándar, por ejemplo de asyncio o del pool) llegan a Sentry sin limpiar: el filtro limpia excepciones y variables, pero no `logentry`. Quitarlo o desactivar la integración de logging, con un test (ahora).
- Una sola IP puede agotar la cuota de Sentry con `/api/public/client-errors`: 30 por hora son unas 21,600 al mes, frente a unas 5,000 del plan gratis (cifra por confirmar). Tope diario global o deduplicar por tipo y ruta (antes de la beta).
- Dos pedidos idénticos simultáneos fallan mal: el segundo pago recibe 409 en vez de 200, dos anulaciones escriben dos auditorías y dos inquilinos con el mismo documento a la vez dan 500. Bloquear la fila del contrato, como pide SPEC y como necesitará CS-15, y la de la cuenta al crear inquilinos (ahora o con CS-15).
- En el contexto del inquilino, las políticas también permiten escribir: un error en un endpoint del portal podría cambiar el RUC del arrendador o insertar cobros. Política RESTRICTIVE que niegue INSERT, UPDATE y DELETE con `app.context = 'tenant'`, salvo en las tablas del portal (antes de CS-18, junto con I5; regla 2).
- El ADR-0007 dice que un texto cifrado no se puede mover a otra fila, pero los datos asociados solo atan tabla, columna y cuenta: el documento de un inquilino se podría copiar a otro de la misma cuenta. Agregar el id de la fila ahora que no hay datos reales; el sobre sigue en `v1` (ahora).
- `casero_app` tiene permisos que no usa: INSERT en `subscriptions` y `subscription_payments`, y UPDATE en todas las columnas de `accounts`, incluidas `purged_at` y las de eliminación. El worker copia los mismos. Reducirlos en la tabla de roles de SPEC y en el test (antes de CS-23; regla 2).
- Cuatro claves (límite de peticiones, hash de correo, baja y aceptaciones) no validan su largo al arrancar, y la del hash de correo es permanente. Exigir 32 bytes, como la del documento (ahora).
- El hash de IP de `legal_acceptances` usa una clave permanente y se guarda para siempre: con la clave, una IPv4 se puede recuperar por fuerza bruta, y la tabla de retención de SPEC no lo menciona. Decidir si se conserva como prueba del consentimiento y se declara, o si se quita (antes de CS-25; regla 2).

Backend

- La renta de un período está implementada dos veces, en Python y en SQL, y no coinciden antes del primer cambio de renta. Tras una corrección del mismo período (CS-31), el aviso de bancarización y la pantalla podrían usar montos distintos. Dejar una sola regla (dentro de CS-31).
- CS-21 puede reutilizar la asignación con tres cambios previos:
  - una lectura en `ledger.application` con la parte excluida de cada pago, el inquilino y el tratamiento;
  - separar en `covered` los ids de pagos y de créditos, que hoy comparten espacio;
  - que `tax` use el `Period` del kernel en vez de texto.

  (antes de CS-21)
- `api.py` tiene 656 líneas, y los validadores (id v7, monto máximo y período) se copian entre módulos. Un módulo compartido de tipos de API y el cableado separado por módulo, con un ADR si cambia AR-03 (antes de CS-15).

Frontend, infraestructura y documentación

- Los tests del frontend siguen reintentando las consultas, y una ruta sin simular responde 404 en silencio (abierto desde la revisión anterior; ahora).

- Los límites de largo muestran el mensaje de zod en inglés ("Too big…") y los campos no tienen `maxLength`: pago, contrato, inmueble y Libro de Reclamaciones (ahora).
- Textos (ahora):
  - la calculadora dice "Septiembre" y la app, "setiembre";
  - la página de inicio sigue titulada "Tu cuenta está lista" encima del panel;
  - "por vencer" y "pendiente" muestran el mismo texto;
  - el error de fecha anterior al control es falso para contratos con saldo anterior.
- Accesibilidad (antes de CS-27):
  - todas las páginas se titulan "Casero" (WCAG 2.4.2);
  - el foco no se mueve al abrir el formulario de pago ni después de guardar o anular;
  - los enlaces de navegación y de "Volver" miden unos 24 px.
- `/app` no tiene estado de carga ni de error: con la API caída, la página de inicio queda en blanco (ahora).
- `formatSoles` falla con negativos ("S/ -1.-50"), y CS-31 trae montos con signo (antes de CS-31).
- Ningún navegador corre con la CSP de producción, así que un problema de CSP (por ejemplo, el registro del service worker de CS-18) aparecería recién en producción. Playwright contra la imagen del borde (antes de CS-18).
- Documentación desactualizada (ahora):
  - las alertas por correo citan `make complaint-copy`, que no existe, y `make load-tax-config`, que solo funciona en local;
  - no hay procedimiento de producción para la beta por invitación;
  - el README no dice que el registro local necesita `make invite`;
  - el árbol de SPEC no tiene `infra/backup` ni `docs/reviews`;
  - el diseño de los respaldos no tiene ADR.

Estado de la revisión anterior

Id	Estado
I1	Abierto (antes de la beta): el inicio de sesión aún espera dentro de la transacción y el correo se envía dentro de ella. I13 lo agrava.
I2	Corregido: `hide_parameters`, manejador global, procesador de structlog y `log_error_verbosity=terse`. Queda el menor del logging estándar.
I3	Corregido y probado en CI: un 502 con `X-Tenant-Token` no deja rastro en los registros de Caddy.
I4	Corregido: `/legal` con `noindex` y test del borde.
I5	Abierto: decisión tuya antes de CS-19, conviene diseñarla junto con las escrituras del inquilino.
I6	Corregido en CS-09.
I7	Abierto: el catálogo sigue mirando solo tablas, sin pertenencias entre roles ni TEMP. Hoy no hay vistas ni secuencias.
I8	Parcial: AR-06, AR-07, AR-08, AR-09, AR-16 y AR-18 siguen igual, sin violaciones actuales. AR-20 empeoró (I12).
I9	Parcial: el correo de privacidad está decidido pero no construido (I20); los marcadores restantes son tuyos.
I10	Corregido: documentado en `server.md` y `backups.md`.
I11	Corregido: puerto `CashThresholds` en CS-12.

Menores	**Corregidos:** • método de redondeo en `tax-config.md`; • deuda del worker (horario de Lima, `job_runs` en `shared`, SIGTERM y candado); • auditoría duplicada; • páginas de error en inglés; • CSP y Sentry (ADR-0011); • PWA asignada a CS-18; • `List-Unsubscribe` movido a CS-34. **Parciales:** • tipo de período compartido (`tax` sigue con texto); • árbol de SPEC. **Abiertos:** • `purge_account` sin verificar la fase de almacenamiento (CS-25); • políticas de cuenta aplicadas a PUBLIC, así que `casero_admin` con `app.account_id` vería filas y el ADR-0002 promete de más (tuyo, antes de CS-24); • `search_path` sin `pg_temp`; • hash de IP del límite de peticiones con clave permanente y fecha, no una sal rotativa (mitigado: fuera de los respaldos y borrado a las 23 horas); • `RucGroup` duplicado; • `api.py`, que creció; • versión en la URL legal (CS-25); • dos políticas de privacidad (tuyo, CS-25); • gracia de la prueba (tuyo, CS-23); • lista de espera a 12 meses (CS-25); • dos límites de 5 MB en Caddy (antes de CS-19); • reintentos en los tests del frontend.

Lo que está sólido

- **RLS y permisos:** todas las tablas de cuenta tienen ENABLE y FORCE, con USING y WITH CHECK, y una política del inquilino que falla cerrada. Los permisos de las migraciones coinciden, rol por rol, con la tabla de SPEC, y hay tests de acceso para las 27 tablas de cuenta.
- **Libro de solo inserción:** no hay UPDATE ni DELETE en las 16 tablas del libro para la app, el worker y el panel. La generación de rentas es idempotente en el dominio y en la base, y no regenera un período anulado.
- **Asignación de pagos:** es una función pura y determinista que cumple la regla "Aplicación" de SPEC. Tiene propiedades de Hypothesis (identidad del saldo, orden de inserción y movimientos posteriores) y casos de febrero, años bisiestos y medianoche UTC.
- **Dinero:** solo enteros, sin float ni división real en el backend ni en la web. Ningún valor de SUNAT está escrito en el código: el umbral llega por el puerto.
- **Documentos del inquilino:** se cifran en el sobre `v1` con HMAC permanente, y un test prueba que el documento no aparece en claro en la base ni en `audit_log`.
- **CI cubre la definición de hecho:** Ruff, Mypy, Pytest con Testcontainers, import-linter (12 contratos), ESLint estricto, tsc, Vitest, build, contrato OpenAPI, auditoría de dependencias, gitleaks, cabeceras del borde con la imagen de producción y una restauración del respaldo de punta a punta.

Plan propuesto

Regla 1, en PR pequeños antes de seguir con el Sprint 7:

1. **Pruebas y transacciones:** AR-20 recursivo con CSRF (I12) y commit antes de la respuesta (I13).

2. **Respaldos y monitoreo:**
   - I14;
   - `/api/health` que consulte la base;
   - heartbeat del worker y alerta de intentos colgados (I15, si no cuesta);
   - limpieza de `logentry`;
   - tope de errores del navegador.
3. **Dinero:**
   - validación de los saldos anteriores (I18);
   - reenvíos simultáneos de pagos y anulaciones;
   - bloqueo al crear inquilinos;
   - la pantalla de pago ante errores (I19), con sus textos.
4. **Web:**
   - tests sin reintentos ni rutas sin simular;
   - mensajes y textos en español;
   - títulos por página, foco y enlaces de 44 px;
   - estados de carga y de error;
   - `formatSoles` con negativos.
5. **Seguridad y documentación:**
   - datos asociados con el id de la fila;
   - largo mínimo de las claves;
   - ADR de respaldos;
   - correcciones de la operación y del árbol de SPEC.
6. **Correo de privacidad (I20):** cableado y test de marcadores.

Con cada historia: la regla única de renta dentro de CS-31; la preparación de CS-21; el módulo compartido de la API antes de CS-15; y Playwright con la CSP de producción antes de CS-18.

Decisiones tuyas (regla 2), con mi recomendación

1. **I16, cobros antes del inicio:** aceptar cobros desde el 1 de enero de 2026 en cualquier contrato, con un aviso si la fecha es anterior al mes de control.
2. **I17, renovación:** agregar "extender el fin previsto" a CS-31 (+1 punto) y consultar al abogado si conviene no cerrar solo.
3. **I15, aviso si el worker se cae:** es decisión tuya solo si el heartbeat del worker tuviera costo.
4. **Escrituras del inquilino:** negarlas por RLS antes de CS-18, diseñadas junto con I5.
5. **Permisos de más:** reducir los de `casero_app` y `casero_worker` antes de CS-23, y crear las políticas de cuenta solo para esos dos roles antes de CS-24.
6. **Hash de IP de las aceptaciones:** conservarlo como prueba del consentimiento y declararlo en la retención y en la política de privacidad, o quitarlo.